INF201 - Lecture 10

9th Nov 2026

Topics

- Object-oriented programming
  - Fundamentals
  - Namespaces
  - Copying items
  - Best practices for writing classes
- Adding command line arguments to your scripts

Object-orientation: Fundamentals

Idea 1: Combine data and operations into new data types

- A (user-defined) data type is a *class*
- Each *object* is an *instance* of a class
- Classes and objects have *attributes* and *methods*
  - *Methods* are functions operating on objects
  - *Attributes* are data (variables) contained in objects

In [1]:
```
from geometries import Circle

c1 = Circle(center=0, radius=1)
type(c1)  # data type: geometries.Circle
```

Out[1]:
```
geometries.Circle
```

In [2]:
```
type(Circle)
```

Out[2]:
```
type
```

In [3]:
```
vars(c1)  # attributes
```

Out[3]:
```
{'ctr': 0, 'rad': 1}
```

Behind the scenes: `vars(c1)` calls the special method `__dict__` of the object `c1`:

In [4]:
```
vars?
```

```
Docstring:
vars([object]) -> dictionary

Without arguments, equivalent to locals().
With an argument, equivalent to object.__dict__.
Type:      builtin_function_or_method
```

In [5]:
```
c1.__dict__
```

Out[5]:
```
{'ctr': 0, 'rad': 1}
```

Note: You should generally not call the special methods (dunders) when you use the classes, they are for internal use only. This is why we would inspect the object attributes by `vars()` instead of `__dict__`.

Idea 2: Allow modification and extension of data types

- *Inheritance*: Define a *subclass* (or child class) based on a *superclass* (or parent class) and adapt or extend it
- Inheritance allows us to write more specialised classes based on general ones
  - Can create deep hierarchies here, but the recommendation is to keep it shallow

Trace the hieracrchy using `__base__`

In [6]:
```
from pets_updated import Dog, Pet

print(Dog.__base__)  # __base__ show us the parent class
```

```
<class 'pets_updated.Pet'>
```

In [7]:
```
print(Pet.__base__)
```

```
<class 'object'>
```

In [8]:
```
print(object.__base__)  # object is the top of the hierarchy
```

```
None
```

Who gets called when?—Method resolution order

Define three classes `A > B > C` where each subclass redefines one method.
(We write this in a compact way, without any doc-strings and type hints for now.)

In [10]:
```
class A:
    def f(self): return "A.f()"
    def g(self): return "A.g()"
    def h(self): return "A.h()"


class B(A):
    def g(self): return "B.g()"


class C(B):
    def h(self): return "C.h()"
```

Visualise this code with Pythontutor

Create one instance per class.

In [11]:
```
a = A()
b = B()
c = C()
```

Call methods on `A` instance

In [12]:
```
print(a.f(), a.g(), a.h())
```

```
A.f() A.g() A.h()
```

Call methods on `B` instance

In [13]:
```
print(b.f(), b.g(), b.h())
```

```
A.f() B.g() A.h()
```

Call methods on `C` instance

In [14]:
```
print(c.f(), c.g(), c.h())
```

```
A.f() B.g() C.h()
```

- Methods are looked up from bottom and up
  1. instance
  2. class
  3. superclasses in ascending order
  4. `object`
- Lookup order for a class is available through the attribute `__mro__` or method `mro()` (method resolution order)

In [15]:
```
A.__mro__
```

Out[15]:
```
(__main__.A, object)
```

In [16]:
```
B.__mro__
```

Out[16]:
```
(__main__.B, __main__.A, object)
```

In [17]:
```
C.__mro__
```

Out[17]:
```
(__main__.C, __main__.B, __main__.A, object)
```

For a more complex example, where a class has two superclasses (parents):

In [18]:
```
class D(A):
    pass


class E:
    def g(self): return "E.g()"
    pass


class F(D, E):
    pass
```

```
f = F()
print(f.f(), f.g(), f.h())
```

```
A.f() A.g() A.h()
```

In [19]:
```
F.__mro__
```

Out[19]:
```
(__main__.F, __main__.D, __main__.A, __main__.E, object)
```

In [20]:
```
class G(B, C):
    pass

g = G()
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
Cell In[20], line 1
----> 1 class G(B, C):
      2     pass
      3
      4 g = G()

TypeError: Cannot create a consistent method resolution order (MRO) for bases B,
C
```

- For multiple inheritance, MRO will be determined like this:
  - A class appears before its parents.
  - Parent classes appear in the order listed.
  - Each class appears only once, preserving relative order.
- If Python cannot determine a consistend MRO, it will fail with a `TypeError`
- `super()` in class definitions refers to the next class in the MRO

Idea 3: Expose an interface, hide the implementation

- *Interface*: Methods "advertised" for public use
  - Provides access to functionality of the class
  - Should be stable over time
  - Should be well documented
  - Should be well tested
- *Implementation*: Does the actual work
  - User should not need to know about it
  - Can change at any time (refactoring)
  - Changes should not affect behavior
- Also know as *encapsulation*

In [21]:
```
from geometries import Rectangle

Rectangle??
```

```
Init signature:
Rectangle(
    lower_left: tuple[int | float, int | float],
    upper_right: tuple[int | float, int | float],
) -> None
Docstring:      <no docstring>
Source:
class Rectangle:
    def __init__(self, lower_left: tuple[int | float, int | float], upper_right:
tuple[int | float, int | float]) -> None:
        """Create rectangle based on lower left and upper rigth coordinates"""
        self.ll = lower_left
        self.ur = upper_right

    def area(self) -> int | float:
        """Calculates the area of the rectangle"""
        return (self.ur[0] - self.ll[0]) * (self.ur[1] - self.ll[1])
File:           ~\documents\inf201-h26-my-repo\lectures\lecture_10\geometries.py
Type:           type
Subclasses:
```

We can change the *implementation* of the rectangle without changing the *interface*

In [22]:
```python
class Rectangle:
    def __init__(self, lower_left: tuple[int | float, int | float], upper_right:
        """Create rectangle based on lower left and upper rigth coordinates"""
        self.width = upper_right[0] - lower_left[0]
        self.height = upper_right[1] - lower_left[1]

    def area(self) -> int | float:
        """Calculates the area of the rectangle"""
        return self.width * self.height
```

We can further protect the height and width attributes, by indicating with `_`

In [23]:
```python
class Rectangle:
    def __init__(self, lower_left: tuple[int | float, int | float], upper_right:
        """Create rectangle based on lower left and upper rigth coordinates"""
        self._width = upper_right[0] - lower_left[0]
        self._height = upper_right[1] - lower_left[1]

    def area(self) -> int | float:
        """Calculates the area of the rectangle"""
        return self._width * self._height
```

... and create properties for viewing the protected attributes

In [24]:
```python
class Rectangle:
    def __init__(self, lower_left: tuple[int | float, int | float], upper_right:
        """Create rectangle based on lower left and upper rigth coordinates"""
        self._width = upper_right[0] - lower_left[0]
        self._height = upper_right[1] - lower_left[1]

    def area(self) -> int | float:
        """Calculates the area of the rectangle"""
        return self._width * self._height

    @property
```

```
    def width(self) -> int | float:
        """View-only property for rectangle width"""
        return self._width

    @property
    def height(self) -> int | float:
        """View-only property for rectangle width"""
        return self._height
```

In [27]:
```
r1 = Rectangle((1, 1), (5, 5))
print(f"Width: {r1.width}")
print(f"Height: {r1.height}")
print(f"Area: {r1.area()}")
```

```
Width: 4
Height: 4
Area: 16
```

Names, Namespaces, and Scopes

Why worry about names?

1. Programs execute functions to manipulate data.
2. Data and functions are stored as sequences of bits in memory.
3. We need *names* to refer to data and functions in our programs.
4. In large programs
   - the same name may be used or different purposes in different places
   - it is impossible to keep an overview over all names
   - E.g.: what do you get if you run `from xyz import *`?
     - (Wildcard imports like this is generally a bad idea, don't do it)
5. Solution: *namespaces* and *scoping rules*

Namespaces (navnerom)

Namespaces help to keep names organized.
A namespace is a 'place' with a set of names and their corresponding objects. (From here)

Scoping rules (regler for gyldighetsområder)

Scoping rules define which namespace applies in each part of a program.

How do we bind names to objects in Python?

Operation | Example | Name bound :- | :- | - Assignment | `x = 2` | `x` Function definition | `def f(): pass` | `f` Class definition | `class A: pass` | `A` Module import | `import math` | `math` || `import math as m` | `m` || `from math import sin` | `sin`

Definitions are statements

Definitions are statements in Python programs. They are executed just as all other statements.

Presentation on topic

- See also https://nedbatchelder.com/text/names1.html

Where are names bound?

```
class Friend:

    greeting = "Hi, "

    def __init__(self, name):
        self.name = name

    def greet(self):
        text = Friend.greeting + self.name
        print(text)

joe = Friend("Joe")
```

Bound in namespace: module, class, function, instance

Name-binding rules

- When a name is bound, it is registered in *exactly one namespace*
- Available namespaces
  - `__builtin__` namespace of Python interepreter
  - each module has a namespace (each imported `*.py` file)
  - each class has a namespace
  - each class *instance* has a namespace
  - each function *invocation* has a namespace
- In which namespace is a name registered?
  - In the namespace of the *innermost scope*
  - Inside a list comprehension: in the comprehension's namespace
  - Inside function definitions: in the function invocation's namespace
  - Inside class definitions: in the class' namespace
  - Otherwise, in the module's namespace

In [28]:
```
ls = [x for x in range(5)]
x  # Only defined within the list comprehension
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[28], line 2
      1 ls = [x for x in range(5)]
----> 2 x  # Only defined within the list comprehension

NameError: name 'x' is not defined
```

In [29]:
```
__builtin__?
```

```
Type:        module
String form: <module 'builtins' (built-in)>
Docstring:
Built-in functions, types, exceptions, and other objects.

This module provides direct access to all 'built-in'
identifiers of Python; for example, builtins.len is
the full name for the built-in function len().

This module is not normally accessed explicitly by most
applications, but can be useful in modules that provide
objects with the same name as a built-in value, but in
which the built-in of that name is also needed.
```

Technically, the Python's built-in functions are all in the builtin module

In [30]:
```
# Jupyter magic for showing all names in current notebook
# List all available magics by %lsmagic
%whos
```

```
Variable    Type         Data/Info
---------------------------------
A           type         <class '__main__.A'>
B           type         <class '__main__.B'>
C           type         <class '__main__.C'>
Circle      type         <class 'geometries.Circle'>
D           type         <class '__main__.D'>
Dog         type         <class 'pets_updated.Dog'>
E           type         <class '__main__.E'>
F           type         <class '__main__.F'>
Pet         type         <class 'pets_updated.Pet'>
Rectangle   type         <class '__main__.Rectangle'>
a           A            <__main__.A object at 0x00000292705338C0>
b           B            <__main__.B object at 0x0000029270533770>
c           C            <__main__.C object at 0x0000029270533A10>
c1          Circle       <geometries.Circle object at 0x0000029270530440>
f           F            <__main__.F object at 0x0000029270533B60>
ls          list         n=5
r1          Rectangle    <__main__.Rectangle object at 0x0000029270731810>
```

In [32]:
```
%lsmagic
```

Out[32]:
```
'Available line magics:\n%alias  %alias_magic  %autoawait  %autocall  %automagi
c  %autosave  %bookmark  %cd  %clear  %cls  %code_wrap  %colors  %conda  %confi
g  %connect_info  %copy  %ddir  %debug  %dhist  %dirs  %doctest_mode  %echo  %e
d  %edit  %env  %gui  %hist  %history  %killbgscripts  %ldir  %less  %load  %lo
ad_ext  %loadpy  %logoff  %logon  %logstart  %logstate  %logstop  %ls  %lsmagic
%macro  %magic  %mamba  %matplotlib  %micromamba  %mkdir  %more  %notebook  %pa
ge  %pastebin  %pdb  %pdef  %pdoc  %pfile  %pinfo  %pinfo2  %pip  %popd  %pprin
t  %precision  %prun  %psearch  %psource  %pushd  %pwd  %pycat  %pylab  %qtcons
ole  %quickref  %recall  %rehashx  %reload_ext  %ren  %rep  %rerun  %reset  %re
set_selective  %rmdir  %run  %save  %sc  %set_env  %store  %subshell  %sx  %sys
tem  %tb  %time  %timeit  %unalias  %unload_ext  %uv  %who  %who_ls  %whos  %xd
el  %xmode\n\nAvailable cell magics:\n%%!  %%HTML  %%SVG  %%bash  %%capture  %%
cmd  %%code_wrap  %%debug  %%file  %%html  %%javascript  %%js  %%latex  %%markd
own  %%perl  %%prun  %%pypy  %%python  %%python2  %%python3  %%ruby  %%script
%%sh  %%svg  %%sx  %%system  %%time  %%timeit  %%writefile\n\nAutomagic is ON,
% prefix IS NOT needed for line magics.'
```

Note that the classes we have defined are listed here, but not the methods inside the classes.
These methods belong to the namespace of the class, not the notebook.

Where are names looked up?

```
class Friend:

    greeting = "Hi, "

    def __init__(self, name):
        self.name = name

    def greet(self):
        text = Friend.greeting + self.name
        print(text)

joe = Friend("Joe")
```

Lookup in: L: local (function), G: global (module), B: builtin

Name-lookup rules

- When we use a name, Python must loop up the name and find the object it refers to
- In which namespace does Python look?
- **LEGB rule** (Mark Lutz, *Learning Python*)
  - **L—Local:** Namespace of the function invocation currently executing
  - **E—Enclosing:** Namespaces of all functions enclosing the definition of the current function (ignore for now)
  - **G—Global:** Namespace of module in which the current function *was defined*
  - **B—Builtin:** Namespace of Python builtins
- Exceptions can be forced with `global` and `nonlocal` keywords (avoid for now)

Attribute/method lookup with "dot"

- Modules, classes, and instances have attributes and methods
- Their names are bound inside module, class, instancance namespace
- Accessible through the dot-operator:

In [33]:
```
class Friend:

    greeting = 'Hi, '

    def __init__(self, name: str) -> None:
        """Create Friend object with attribute name"""
        self.name = name

    def greet(self) -> None:
        """Print string greeting self"""
        print(Friend.greeting + self.name)


joe = Friend("Joe")
print(joe.name)
```

```
Joe
```

In [34]:
```
joe.name = "Joe Doe"  # change the name attribute
joe.greet()
```

```
Hi, Joe Doe
```

In [35]:
```
Friend.greeting = "Hello, "  # change the greeting in the class Friend
joe.greet()
```

```
Hello, Joe Doe
```

Attribute lookup in instances vs classes

```
class Friend:
    def __init__(self, name):
        self.name = name

    def greet(self):
        print("Hi, " + self.name)

joe = Friend("Joe")
print(joe.name)
joe.greet()
```

Found in: instance, class

Pitfall: Duplicate attribute names

- Python uses the same namespace for methods and data attributes
- Names are looked up in the instance namespace first, then in the class namespace
- This may lead to surprises when using the same name in multiple places

In [44]:
```
class Friend:

    def __init__(self, name: str) -> None:
        """Create Friend with name"""
        self.name = name

    def greet(self) -> None:
        """Print greeting"""
        print('Hi,', self.name)

    def print_name(self) -> None:
        """Print statement of name"""
        print('Your name is', self.name)
```

In [45]:
```
joe = Friend('Joe')
joe.greet()
```

```
Hi, Joe
```

In [46]:
```
joe.print_name()
```

```
Your name is Joe
```

- What happended here?
  - Lookup starts with the instance, where we find the string stored in `self.name``

- Then Python tries to call this string as a function because of the `()` after `joe.name`
- Lookup stops at first match and thus never sees the method `name(self)` in this case

Visualised with Pythontutor

Copying objects

Objects with custom classes comes with the same potential pitfall for copying that objects of built-in classes (such as lists) do.

In [47]:
```
class ClassName:
    pass


a = ClassName()
a.x = 10  # We create an instance attribute
b = a  # a and b are two names for the same object
b.x = 20
print(a.x, b.x)
```

```
20 20
```

- `a` and `b` refer to the same object, so changing `b` also changes `a`
- Visualise this with PythonTutor

Sidenote: If you want to prevent the user from setting instance attributes not defined in the class you can use `__slots__`.
Read more about them in Python's documentation on slots and in chapter 3 of Advanced Guide to Python 3 Programming.

Python's `copy` module

- Provides functions to copy objects

In [48]:
```
import copy

c = copy.copy(a)
c.x = 50
print(a.x, b.x, c.x)
```

```
20 20 50
```

In [49]:
```
s = ClassName()
s.m = 'Nice weather today.'
t = copy.copy(s)
print(hex(id(s)), hex(id(t)))
print(hex(id(s.m)), hex(id(t.m)))
```

```
0x292707316d0 0x2927058b490
0x29270790b70 0x29270790b70
```

- `s` and `t` have *different* `id()`: they are *different* `S` *instances*
- `s.m` and `t.m` have the same `id()`: they are the *same string instance*
- `copy.copy()` is a *shallow copy*: `t` is a new instance with its own namespace, but the names refer to the same objects as in `s`
- Assignment to a member re-binds the name to a new string object

In [50]:
```
t.m = 'The forecast for tomorrow is also nice.'
print(hex(id(s)), hex(id(t)))
print(hex(id(s.m)), hex(id(t.m)))
```

```
0x292707316d0 0x2927058b490
0x29270790b70 0x29270a26560
```

Copying object with mutable members: deep copy

- Mutables: lists, dictionaries, objects of most classes
- What does shallow copy mean for objects with mutable members?

In [51]:
```
u = ClassName()
u.m = [1, 2, 3]
v = copy.copy(u)
u.m.append(4)
print(u.m, v.m)
print(hex(id(u.m)), hex(id(v.m)))
```

```
[1, 2, 3, 4] [1, 2, 3, 4]
0x29270a3a5c0 0x29270a3a5c0
```

- Lists are changed in *both* `u` and `v` because their `m` refers to the same list object
- Solution: *deep copy* (as we used for lists earlier)

In [52]:
```
w = copy.deepcopy(u)
print(u.m, v.m, w.m)
print(hex(id(u.m)), hex(id(v.m)), hex(id(w.m)))
```

```
[1, 2, 3, 4] [1, 2, 3, 4] [1, 2, 3, 4]
0x29270a3a5c0 0x29270a3a5c0 0x29270a0f7c0
```

Forgetting the difference between shallow and deep copy is a common source of errors in Python programs!

Recommendations and best practices for writing classes

- Use doc-strings and type hints
  - The same way as you would for functions
- Keep clases small
  - Each class should have a single responsibility
- Place class attributes at the top, followed by `__init__`, then public methods, then private helpers
- The initialiser should not allow you to create an object with "illegal" attributes

    - For example: a circle with a negative radius, a dice with only one side
- Implement methods or overload existing methods only when it makes sense
    - For example: only implement `__lt__` (less than) and other comparisons if there is a logical way to sort objects of this class
    - Most classes should (at least) have `__init__` and `__repr__` (help with debugguing)
- Attributes that the user does not need to access, should be protected
    - If you want a viewer or setter for the protected attribute, use the `@property` decorator
- Keep inheritance hieracrhies shallow
- If both options make sense: Chose composition over inheritance

Including command-line options

- Sometimes you want to be able to run your code from the command-line
    - For example: You are training an AI algorithm on a supercomputer that only has command-line
    - Or maybe you are running a simulation and want to test it with lots of different stating points
- In such cases it is useful to be able to pass the needed arguments using the command-line

```
python my_script.py --one_argument 10 --other_argument "something"
```

- Python has a built-in package `argparse` for including command-line arguments
    - argparse
    - Tutorial
- Overall idea
    - Build a parser by adding options we want to support
    - Can also provide default values and help texts
    - Then run parser to evaluate command line arguments
    - Finally extract values from parser
- Let's look at `demo.py` for a very simple example
- We exploit object-orienter programming here
    - We create an instance of class `ArgumentParser``
    - We use its `add_argument()` method to add the options we want
    - We use its `parse_args()` method to analyse the command line and get and `args` instance containing all options
    - We can now look up the options as fields of `args`, e.g., `args.name``

In [53]:
```
import argparse

argparse.ArgumentParser?
```

```
Init signature:
argparse.ArgumentParser(
    prog=None,
    usage=None,
    description=None,
    epilog=None,
    parents=[],
    formatter_class=<class 'argparse.HelpFormatter'>,
    prefix_chars='-',
    fromfile_prefix_chars=None,
    argument_default=None,
    conflict_handler='error',
    add_help=True,
    allow_abbrev=True,
    exit_on_error=True,
    *,
    suggest_on_error=False,
    color=True,
)
Docstring:
Object for parsing command line strings into Python objects.

Keyword Arguments:
    - prog -- The name of the program (default:
        ``os.path.basename(sys.argv[0])``)
    - usage -- A usage message (default: auto-generated from arguments)
    - description -- A description of what the program does
    - epilog -- Text following the argument descriptions
    - parents -- Parsers whose arguments should be copied into this one
    - formatter_class -- HelpFormatter class for printing help messages
    - prefix_chars -- Characters that prefix optional arguments
    - fromfile_prefix_chars -- Characters that prefix files containing
        additional arguments
    - argument_default -- The default value for all arguments
    - conflict_handler -- String indicating how to handle conflicts
    - add_help -- Add a -h/-help option
    - allow_abbrev -- Allow long options to be abbreviated unambiguously
    - exit_on_error -- Determines whether or not ArgumentParser exits with
        error info when an error occurs
    - suggest_on_error - Enables suggestions for mistyped argument choices
        and subparser names (default: ``False``)
    - color - Allow color output in help messages (default: ``False``)
File:           ~\appdata\local\python\pythoncore-3.14-64\lib\argparse.py
Type:           type
Subclasses:     ArgumentParser, _KVArgParser, MagicArgumentParser
```

Testing Python code

> Tests are the dental floss of development: everyone knows they should do it more, but they don't, and they feel guilty about it.

From: Getting Started Testing by Ned Batchelder

- Systematic testing is essential part of quality control
- Do not trust code that comes without tests!
- But: passings tests are no guarantee that everything is correct
  - Tests might not cover all code

    - Tests may not cover all possible situations
    - Tests may pass for the wrong reasons
- Different levels of testing
    - Unit tests: test "units", i.e., functions and methods
    - Integration tests: test larger parts, e.g., modules or packages
    - Acceptance tests: tests by client required for accepting delivery, test the entire system against client use-cases
    - Regression tests: test detecting bugs in earlier versions, kept to avoid falling back to old mistakes

Agile software development

- Agile software development is a modern (2001-) set of software development methods
- Focus on quick delivery, frequent updates, and flexibility, while maintaining quality
- Strong focus on testing
- Test-driven development is part of Agile
    - Write tests first, otherwise you'll never write them
    - Writing tests for working code often results in weak tests
    - Therefore
        1. Write test
        2. Write code only when a test fails
        3. Write code until tests passes (and no more)
        4. Review and refactor code without breaking tests
        5. Go back to 1
    - Important: Always run all tests after code changes
- Advantages
    - We can be much more confident in our code
    - We can make changes, big and small, and immediately check that the code still works correctly

Python testing in VS Code